Table of Contents

Introduction **4**
A Practical C Program: Process Information **4**
The C Standard Library: Two Halves **5**
Part 1: Header Files 5
Part 2: Precompiled Machine Code 6
Summary: The Two Part Anatomy 7
Static VS Dynamic Linking **7**
i) Static Linking 7
ii) Dynamic Linking 7
Why it Matters 8
Inspecting Executables with readelf **8**
Three Views of an Executable 8
View 1: The Elf Header (readelf -h) 9
View 2: The Program Headers (readelf -l) 9
View 3: The Dynamic Section (readelf -d) 10
Tracing Dependencies with LDD **11**
Static Binaries and ldd 11
Command Reference Summary **12**
Conclusion **13**
Knowledge Check Questions **14**

List of Figures

Figure I: Program to observe process information.....4
Figure 2: .c file compiled and executed.....5
Figure 3: Linux command (ls, grep) to verify physical location and (stdio.h, unstid.h) to inspect the contents of C Standard Library header files.....6
Figure 4: Linux command to verify presence of “libc”.....6
Figure 5: gcc -static compiling procinfo.c into procinfo_static. The resulting binary is large (~961K) because all library code is embedded directly inside the executable.....7
Figure 6: “gcc procinfo.c -o procinfo_dynamic”, producing a small binary (~16K) because it uses shared libraries loaded at runtime instead of embedding all code.....8
Figure 7: Linux command to view properties/metadata of an executable file “procinfo_dynamic”... 9
Figure 8: Linux command to view program headers, memory layout and interpreters of an executable file “procinfo_dynamic”.....10
Figure 9: Linux command to list dynamic section entries of the executable file “readelf -d procinfo_dynamic”, including required shared libraries (libc.so.6) and initialization/finalization routines.....11
Figure 10: Dynamic executable dependencies shown with ldd.....11
Figure 11: Static executable reports “not a dynamic executable.”.....11
Figure 12: Dynamic section showing ELF bookmarks (references) like (NEEDED) libc.so.6, (INIT), (FINI) that guide the dynamic linker. 15

List of Tables

Table I: Two Parts of the C Standard Library..... 7
Table II: Command Reference: Inspecting ELF Executables.....12

Introduction

When a programmer writes a C program that uses library functions like printf(), getpid(), and time(), compiles it, and runs it, a remarkable chain of events occurs. The compiler transforms human-readable C code into machine instructions. The linker resolves references to external libraries like libc. The operating system reads the resulting binary and maps it into virtual memory. This lab answers three concrete questions:
1) How does a C program link to the Standard Library?
2) Where do C library headers and compiled code physically live on disk?
3) What is the difference between static and dynamic linking, and how can we inspect the resulting executables?

A Practical C Program: Process Information

We begin with a C program that demonstrates real library usage by displaying process information. This program uses multiple functions from the C Standard Library and POSIX extensions:

```
  GNU nano 8.7.1                procinfo.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <time.h>
int main(void) {
        // Get current process ID
        pid_t my_pid = getpid();
        pid_t my_ppid = getppid();

        // Get current time
        time_t current_time = time(NULL);
        struct tm *time_info = localtime(&current_time);

        // Format and display information
        printf("=== Process Information ===\n");
        printf("My Process ID (PID): %d\n", my_pid);
        printf("Parent Process ID: %d\n", my_ppid);
        printf("Current Time: %s", asctime(time_info));
        printf("Executable Path: /proc/self/exe\n");

        return 0;
}
```

Figure I: Program to observe process information

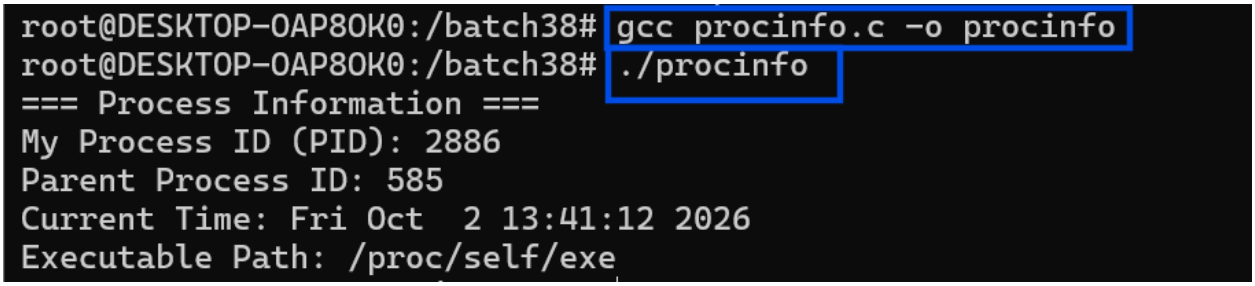


Figure 2: .c file compiled and executed

This program uses several library functions:
• getpid() and getppid(),
• time() and localtime(),
• printf() and asctime().
All of these functions are implemented in the C Standard Library (libc). But where does the actual machine code for these functions come from? The real machine code for functions like printf(), getpid(), and time() lives in the C Standard Library (libc).
On Linux, this is glibc, stored as:

- Static libraries (.a) → /usr/lib/x86_64-linux-gnu/libc.a
- Shared libraries (.so) → /lib/x86_64-linux-gnu/libc.so.6

The C Standard Library: Two Halves

Every C program implicitly depends on the C Standard Library, commonly called libc. On Linux, the GNU implementation is glibc. This library is split into two distinct physical components that serve fundamentally different purposes.

Part 1: Header Files

The Construction Manual Header files (.h) contain declarations: function prototypes, macro definitions, type definitions (typedef), and structure definitions. They tell the compiler what functions exist and how to call them, but they contain no executable machine code. Analogy: A header file is like a construction manual. It tells the architect (the compiler) the dimensions and interface of every component, but it does not contain the actual bricks and steel.

Physical location: /usr/include/

We can verify this on linux system:

```
root@DESKTOP-OAP8OK0:/# ls /usr/include/stdio.h
/usr/include/stdio.h
root@DESKTOP-OAP8OK0:/# ls /usr/include/unistd.h
/usr/include/unistd.h
root@DESKTOP-OAP8OK0:/#  grep "printf" /usr/include/stdio.h | head -2
/* Maximum length of printf output for a NaN.  */
extern int fprintf (FILE *__restrict __stream,
root@DESKTOP-OAP8OK0:/# grep "getpid" /usr/include/unistd.h | head-2
extern __pid_t getpid (void) __THROW;
```

Figure 3: Linux command (ls, grep) to verify physical location and (stdio.h, unstid.h) to inspect the contents of C Standard Library header files.

When you write #include , the preprocessor literally copies the contents of this file into your source code before compilation begins. The compiler now knows that printf() exists and what arguments it expects, but it does not yet know where the actual code lives.

Part 2: Precompiled Machine Code

The Actual Libraries The actual compiled machine instructions for functions like printf(), getpid(), time(), and malloc() live in library files. These come in two forms:
• Static libraries (.a): Archives of object files. The linker copies the needed code directly into your executable.
• Shared/Dynamic libraries (.so): Standalone binaries that are loaded into memory at runtime by the dynamic linker. Multiple programs share a single copy in RAM.

Physical locations:
/lib/x86_64-linux-gnu/
/usr/lib/x86_64-linux-gnu/
/usr/lib/
/lib/

We can verify the presence of “libc”:

```
root@DESKTOP-OAP8OK0:/# ls /lib/x86_64-linux-gnu/libc.so*
/lib/x86_64-linux-gnu/libc.so  /lib/x86_64-linux-gnu/libc.so.6
root@DESKTOP-OAP8OK0:/# ls /usr/lib/x86_64-linux-gnu/libc.a
/usr/lib/x86_64-linux-gnu/libc.a
```

Figure 4: Linux command to verify presence of “libc”.

Summary: The Two Part Anatomy

Component	Contains	Location
Headers (.h)	Declarations, prototypes, macros	/usr/include/
Static lib (.a)	Precompiled object code (archive)	/usr/lib/
Shared lib (.so)	Precompiled position-independent code	/lib/

Table I: Two Parts of the C Standard Library

Static VS Dynamic Linking

Linking is the process of resolving symbolic references (e.g., the name "printf") to actual memory addresses. The timing and method of this resolution define two fundamentally different approaches.

i) Static Linking

In static linking, the linker (ld) copies the required machine code from .a archive files directly into the final executable at compile time.

```
root@DESKTOP-OAP8OK0:/batch38# gcc -static procinfo.c -o procinfo_static
root@DESKTOP-OAP8OK0:/batch38# ls -lh procinfo_static
-rwxr-xr-x 1 root root 961K Oct  2 22:10 procinfo_static
```

Figure 5: gcc -static compiling procinfo.c into procinfo_static. The resulting binary is large (~961K) because all library code is embedded directly inside the executable.

Characteristics:
• The executable is self-contained; it needs no external .so files to run.
• The binary is significantly larger (850KB vs 16KB) because it embeds all library code.
• Updating a library requires recompiling every program that uses it.
• Useful for embedded systems, containers, and environments where portability is critical.

ii) Dynamic Linking

In dynamic linking, the executable contains only references to shared libraries. The actual code is loaded into memory at runtime by the dynamic linker (typically /lib64/ld-linux-x86-64.so.2).

```
root@DESKTOP-OAP8OK0:/batch38# gcc procinfo.c -o procinfo_dynamic
root@DESKTOP-OAP8OK0:/batch38# ls -lh procinfo_dynamic
-rwxr-xr-x 1 root root 16K Oct  2 22:25 procinfo_dynamic
```

Figure 6: "gcc procinfo.c -o procinfo_dynamic", producing a small binary (~16K) because it uses shared libraries loaded at runtime instead of embedding all code.

Why it Matters

Dynamic linking reduces memory consumption and disk usage across the entire system. However, it introduces runtime dependencies and potential version conflicts ("dependency hell"). Static linking eliminates these problems at the cost of larger binaries and no automatic security patching.

Inspecting Executables with readelf

On Linux, compiled executables use the ELF (Executable and Linkable Format) binary format. The readelf tool allows us to read the internal metadata of an ELF file without executing it.

Three Views of an Executable

An ELF binary can be understood through three complementary views:
1) ELF Header (readelf-h): Global metadata about the file—is it a valid executable, and for which CPU?
2) Program Headers (readelf-l): Instructions for the OS loader—which segments to map into memory and with what permissions.
3) Dynamic Section (readelf-d): Metadata for the dynamic linker—which shared libraries must be loaded.

View 1: The Elf Header (readelf -h)

```
root@DESKTOP-OAP8OK0:/batch38# readelf -h procinfo_dynamic
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x1140
  Start of program headers:          64 (bytes into file)
  Start of section headers:          14264 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         14
  Size of section headers:           64 (bytes)
  Number of section headers:         31
  Section header string table index: 30
```

Figure 7: Linux command to view properties/metadata of an executable file "procinfo_dynamic".

Key fields:
• Magic: The bytes 7f 45 4c 46 (".ELF") identify this as a valid ELF file.
• Class: ELF64 means 64-bit.
• Machine: Must match the host CPU. An ARM binary will not execute on x86 64.
• Type: EXEC (static) or DYN (position-independent, used for both shared libraries and modern dynamically-linked executables).

View 2: The Program Headers (readelf -l)

The Program Headers describe segments—contiguous regions of the file that the OS loader maps into virtual memory.

```
root@DESKTOP-OAP8OK0:/batch38# readelf -l procinfo_dynamic

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x1140
There are 14 program headers, starting at offset 64

Program Headers:
  Type           Offset             VirtAddr           PhysAddr
                 FileSiz            MemSiz              Flags  Align
  PHDR           0x0000000000000040 0x0000000000000040 0x0000000000000040
                 0x0000000000000310 0x0000000000000310  R      0x8
  INTERP         0x0000000000000374 0x0000000000000374 0x0000000000000374
                 0x000000000000001c 0x000000000000001c  R      0x1
      [Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
  LOAD           0x0000000000000000 0x0000000000000000 0x0000000000000000
                 0x00000000000007b8 0x00000000000007b8  R      0x1000
  LOAD           0x0000000000001000 0x0000000000001000 0x0000000000001000
                 0x0000000000000311 0x0000000000000311  R E    0x1000
```

Figure 8: Linux command to view program headers, memory layout and interpreters of an executable file "procinfo_dynamic".

Critical entries to look for:

• INTERP: Specifies the path to the dynamic linker (e.g., /lib64/ld-linux-x86-64.so.2). The ker nel reads this and loads the dynamic linker before the program's main() runs

• LOAD: Each LOAD segment corresponds to a memory region:

  – R only: typically .rodata (read-only constants).

  – R E: the .text segment (executable machine code).

  – RW: the .data and .bss segments (read-write global variables).

• GNU_STACK: Declares the stack's permissions. RW means the stack is readable and writable but not exe cutable—this is the secure default that prevents stack based buffer overflow exploits. If this shows RWE, the stack is dangerously executable.

View 3: The Dynamic Section (readelf -d)

This section exists only in dynamically-linked executables. It tells the dynamic linker which shared libraries to load and where to find symbols.

```
root@DESKTOP-OAP8OK0:/batch38# readelf -d procinfo_dynamic

Dynamic section at offset 0x2d90 contains 27 entries:
  Tag        Type                         Name/Value
 0x0000000000000001 (NEEDED)             Shared library: [libc.so.6]
 0x000000000000000c (INIT)               0x1000
 0x000000000000000d (FINI)               0x1304
```

Figure 9: Linux command to list dynamic section entries of the executable file "readelf -d procinfo_dynamic", including required shared libraries (libc.so.6) and initialization/finalization routines.

Critical entries:
• NEEDED: Lists every shared library the program requires. The dynamic linker will search for and mmap() each of these into the process's address space before main() begins. If any NEEDED library is missing, the program will refuse to start.

Tracing Dependencies with LDD

While readelf -d shows the names of required libraries, ldd goes further: it actually invokes the dynamic linker to resolve each NEEDED entry to a full filesystem path.

```
root@DESKTOP-OAP8OK0:/batch38# ldd procinfo_dynamic
        linux-vdso.so.1 (0x0000798b7c390000)
        libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x0000798b7c000000)
        /lib64/ld-linux-x86-64.so.2 (0x0000798b7c392000)
```

*Figure 10: Dynamic **executable dependencies shown with** `ldd`.*

What this tells us:
• libc.so.6 is found at /lib/x86_64-linux-gnu/libc.so.6.
• linux-vdso.so.1 is a virtual shared object injected by the kernel into every process. It provides fast system call entry points without a full context switch.
• /lib64/ld-linux-x86-64.so.2 is the dynamic linker itself—the program that reads the INTERP and NEEDED entries and loads everything.

Static Binaries and ldd

If we run ldd on a statically-linked binary, the output confirms that no dynamic dependencies exist:

```
root@DESKTOP-OAP8OK0:/batch38# ldd procinfo_static
        not a dynamic executable
```

Figure 11: Static executable reports "not a dynamic executable."

This is because the static binary has no “DYNAMIC” section and no “INTERP” entry in its program headers.

Command Reference Summary

Table II consolidates the key tools, the specific fields to inspect, and what each field tells the operating system.

Command / Term	Look For	What It Tells the OS
`readelf -h <file>`	`Type`, `Machine`	“Is this a valid executable, and does it match my CPU architecture?”
`readelf -l <file>`	`INTERP`	“Which dynamic linker must run first to handle shared libraries?”
`readelf -l <file>`	`GNU_STACK`	“Is this stack secure (`RW`) or dangerously executable (`RWE`)?”
`readelf -l <file>`	`LOAD` (Flags)	“Which memory regions are code (`R E`), read-only data (`R`), or read-write data (`RW`)?”
`readelf -d <file>`	`NEEDED`	“What external shared libraries must be mapped into memory before `main()` starts?”
`ldd <file>`	Full output	“Where exactly on the filesystem are the required `.so` files located?”
`ldd <file>`	“not a dynamic executable”	“This binary is statically linked and has no runtime library dependencies.”

Table II: Command Reference: Inspecting ELF Executables

Conclusion

This lab examined how a practical C program using multiple library functions (printf, getpid, time) links to the C Standard Library, which consists of header files in /usr/include/ (the construction manual) and precompiled machine code in /lib/ and /usr/lib/ (the actual libraries). We contrasted static linking, which produces self-contained but large binaries, with dynamic linking, which produces small executables that share library code in RAM at runtime. We used readelf and ldd to inspect the three critical views of an ELF executable: the ELF Header (validity and architecture), the Program Headers (memory layout and security), and the Dynamic Section (runtime dependencies).

Knowledge Check Questions

1) What is the fundamental difference between a C header file and a shared library?
Ans: C header files Header files(.h) are like manuals. They tell the compiler what functions exist (printf, getpid, etc.), but contain no machine code.
While shared library (.so) = shared/dynamic library (loaded at runtime, shared across programs).

---

2) What is static and dynamic linking?
Ans: Static linking → program contains all needed code inside itself. It's characteristics are:

- Bigger file size.
- No external dependency at runtime.
- Must recompile if library updates.

Dynamic linking → program only stores references; actual code is loaded from shared libraries at runtime. It's characteristics are:

- Smaller file size.
- Saves memory (libraries shared).
- Easier updates (library patch benefits all programs).
- Needs .so files present at runtime.

Why It Matters is because:
Dynamic linking reduces memory consumption and disk usage across the entire system. However, it introduces runtime dependencies and potential version conflicts ("dependency hell"). Static linking eliminates these problems at the cost of larger binaries and no automatic security patching.

---

3) What is ELF Header? Write down the commands to check headers.
Ans: ELF Header (readelf -h): Global metadata about the file i.e at start of executable file (is it a valid executable, and for which CPU?. The command to check headers are:
readelf -h <executable filename> (Eg: readelf -h task1) (lets us inspect it's validity, architecture and type)

---

4) What does bookmark mean? provide an example.
Ans: Bookmark = a saved reference or pointer to a location.
Example (ELF context): In readelf -d procinfo_dynamic, entries like (NEEDED) libc.so.6 act as bookmarks telling the dynamic linker "go load this library."
Example (general): In a browser, bookmarking https://www.google.com lets you quickly return to Google later.

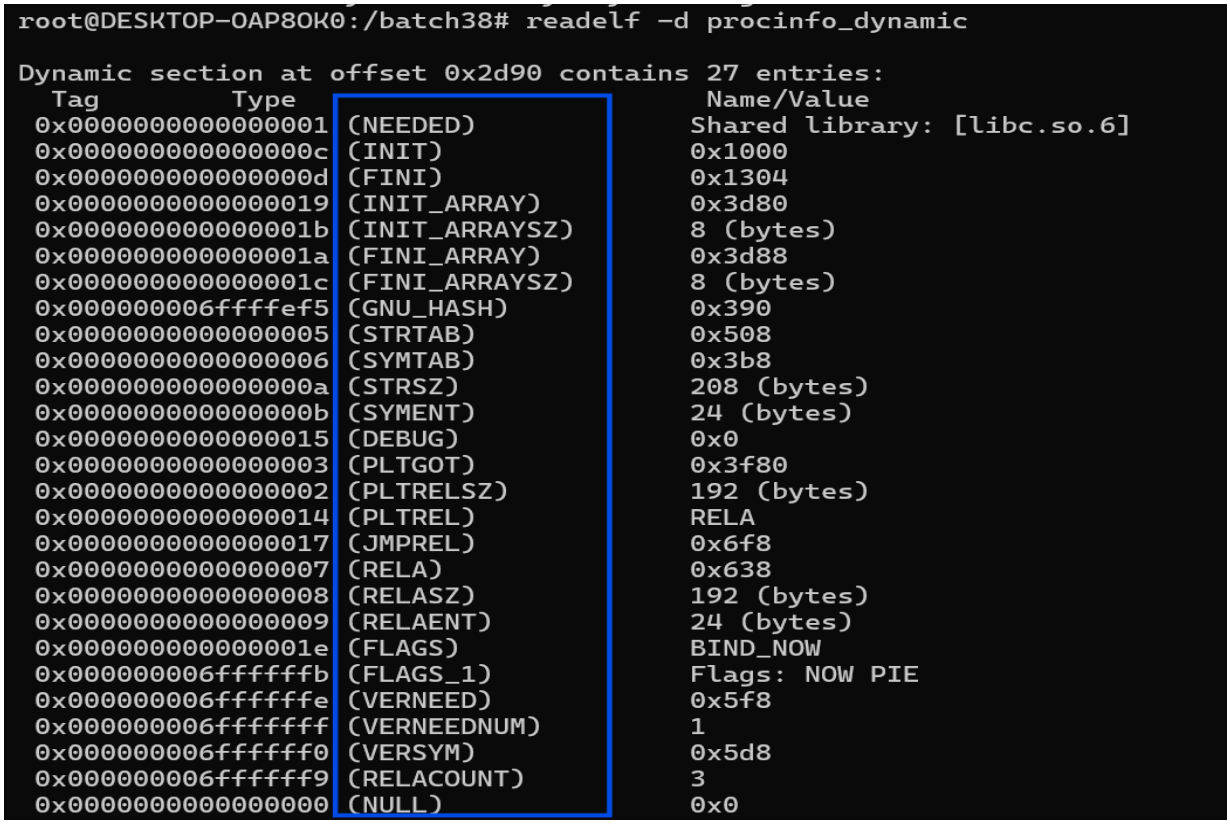


Figure 12: Dynamic section showing ELF bookmarks (references) like (NEEDED) libc.so.6, (INIT), (FINI) that guide the dynamic linker.

Github link: [OS-Security-And-Networks/Week2/Lab_2.1 Task .pdf at main · firstnamecheten/OS-Security-And-Networks]